

School Management App

Frontend Alapok – Axios, Auth, React Query, Forms

React 18 + Laravel 12 API | 2026

1. A választott eszköztár és indoklás

A frontend fejlesztéshez tudatosan választott eszközkombinációt használunk. A cél: minél kevesebb boilerplate kód, tiszta felelősség-elválasztás, és olyan könyvtárak amik pontosan arra a problémára valók amit megoldanak.

Eszköz	Felelőssége	Miért ezt és nem más?
Axios	HTTP kérések küldése	Interceptor rendszer – token automatikus csatolás
React Context API	Globális UI állapot (user, token)	Natív React, nem kell extra csomag
TanStack Query	Szerver adatok (cache, loading, error)	Sokkal kevesebb kód mint Redux, automatikus cache
React Hook Form	Form állapot és validáció	Kevesebb re-render, egyszerű validációs szintaxis
React Router	Oldalak közti navigáció	De facto standard React routinghoz
Bootstrap 5	UI stílusok	Már ismert, gyors fejlesztés

Miért nem Redux?

Redux akkor éri meg igazán, ha bonyolult KLIENS oldali állapot van (form wizard, drag-and-drop). Ebben a projektben az állapot 95%-a SZERVER adat (diákok, tanárok, vizsgák) – erre a React Query kifejezetten jobb eszköz: automatikus cache-elés, automatikus frissítés mutáció után, sokkal kevesebb kód (nincs action, reducer, dispatch).

2. Axios – HTTP kliens interceptorokkal

Az Axios instance-t egy KÖZPONTI helyen hozzuk létre (api/axios.js), nem közvetlenül importáljuk az Axios csomagot mindenhol. Ez teszi lehetővé az interceptorok egységes alkalmazását.

2.1 Az Axios instance létrehozása

```
const api = axios.create({
  baseURL: '/api',
  headers: {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
  },
});
```

Miért saját instance és nem közvetlen axios.get()?

Ha mindenhol `axios.get('http://localhost/api/students')`-t írnánk, minden módosításnál (pl. base URL változás production-ben) az egész kódbázisban kellene keresgélni. Egy központi instance-szal (`api.get('/students')`) elég EGY helyen módosítani.

2.2 Request interceptor – token automatikus csatolása

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');

  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }

  return config; // KÖTELEZŐ! Enélkül a kérés nem indul el.
});
```

KRITIKUS: mindig return config; a végén!

Ez volt a projekt egyik legfontosabb hibája fejlesztés közben: ha az interceptor nem adja vissza a config objektumot, az Axios undefined-et kap, és CSENDBEN nem küldi el a kérést – nincs hálózati hívás a Network fülön, nincs explicit hibaüzenet a konzolon. Minden interceptornak (request és response is) explicit return-nel kell zárnia.

2.3 Response interceptor – automatikus kijelentkeztetés

```
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      localStorage.removeItem('user');
      window.location.href = '/login';
    }

    return Promise.reject(error);
  }
);
```

```
}  
);
```

Miért itt kezeljük a 401-et és nem minden komponensben külön?

Ha a token lejár vagy érvénytelen, a Laravel MINDEN védett végponton 401-et ad vissza. Ahelyett hogy minden egyes API hívásnál külön try-catch-ben kezelnénk ezt, EGY helyen (az interceptorban) figyeljük – ha bárhol jön 401, automatikusan kijelentkeztetünk. Ez DRY elv (Don't Repeat Yourself).

3. React Context API – globális auth állapot

A Context API a bejelentkezett felhasználót és a hozzá tartozó műveleteket (login, logout) osztja meg az egész alkalmazásban, anélkül hogy minden komponensnek props-ként kellene továbbadni (ezt hívják 'prop drilling'-nek).

3.1 Miért van szükség Context-re?

```
// Context NÉLKÜL - prop drilling  
<App user={user}>  
  <Layout user={user}>  
    <Navbar user={user}>  
      <UserMenu user={user} /> // végre itt kell!  
    </Navbar>  
  </Layout>  
</App>  
  
// Context-tel - bármelyik komponens közvetlenül elérí  
function UserMenu() {  
  const { user } = useAuth(); // nincs prop drilling  
}
```

3.2 AuthProvider felépítése

```
export const AuthContext = createContext(null);  
  
export function AuthProvider({ children }) {  
  const [user, setUser] = useState(null);  
  const [loading, setLoading] = useState(true);  
  
  useEffect(() => {  
    const token = localStorage.getItem('token');  
    if (token) {
```

```

        api.get('/auth/me')
            .then((response) => setUser(response.data.data))
            .catch(() => localStorage.removeItem('token'))
            .finally(() => setLoading(false));
    } else {
        setLoading(false);
    }
}, []);

async function login(email, password) {
    const response = await api.post('/auth/login', { email, password
});
    localStorage.setItem('token', response.data.token);
    setUser(response.data.user);
    return response.data.user;
}
}

```

Miért kell a `useEffect` az oldal betöltésekor?

Amikor az oldal újratöltődik (F5), a React állapot (`useState`) TÖRLŐDİK – de a token megmarad a `localStorage`-ban. A `useEffect` ellenőrzi van-e mentett token, és ha igen, lekéri a felhasználó adatait az `/auth/me` végponttól, hogy a bejelentkezés 'megmaradjon' F5 után is.

Miért van 'loading' állapot is, nem csak 'user'?

Az `/auth/me` lekérdezés ASZINKRON – amíg fut, nem tudjuk még hogy a felhasználó be van-e jelentkezve vagy sem. Enélkül a `ProtectedRoute` rögtön a login oldalra irányítana át minden F5 után, MIELŐTT az `/auth/me` válasza megérkezne.

3.3 `useAuth` hook – kényelmi wrapper

```

export function useAuth() {
    const context = useContext(AuthContext);

    if (!context) {
        throw new Error('useAuth csak az AuthProvider-en belül használható!');
    }

    return context;
}

```

Miért nem közvetlenül `useContext(AuthContext)`-et hívunk mindenhol?

Rövidebb és olvashatóbb, és a hibakezelés egy helyen van – ha valaki az `AuthProvider`-en KÍVÜL próbálja használni, egyértelmű hibaüzenetet kap egy rejtélyes hiba helyett.

4. TanStack Query (React Query) – szerver állapot kezelés

A React Query kezeli az API-ból érkező adatok cache-elését, betöltési állapotát és hibakezelését – automatikusan, jóval kevesebb kóddal mint amit useState + useEffect kombinációval kellene írni.

4.1 QueryClient beállítása

```
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 30 * 1000,
      retry: 1,
    },
  },
});
```

Mi az a staleTime?

Alapértelmezetten a React Query minden alkalommal újra lekéri az adatot. A staleTime azt mondja meg meddig tekintse frissnek az adatot: eddig nem kérdezi le újra feleslegesen.

4.2 Egy Query hook felépítése

```
export function useStudents() {
  return useQuery({
    queryKey: ['students'],
    queryFn: () => api.get('/students').then((res) => res.data),
  });
}

function StudentList() {
  const { data, isLoading, isError } = useStudents();
  if (isLoading) return <p>Betöltés...</p>;
  if (isError) return <p>Hiba történt!</p>;
  return <ul>{data.data.map(s => <li key={s.id}>{s.name}</li>)}</ul>;
}
```

5. React Hook Form – form kezelés

A React Hook Form minimalizálja a re-renderek számát gépeléskor, és egyszerű, deklaratív validációs szintaxist ad.

5.1 register() – mező bekötése validációval

```
<input
  type='email'
  className={`form-control ${errors.email ? 'is-invalid' : ''}`}
  {...register('email', {
    required: 'Az email cím megadása kötelező.',
  })}
/>
```

Mit csinál a spread szintaxis?

A register() egy objektumot ad vissza (name, onChange, onBlur, ref) – ezeket a spread ráteríti az input-ra. Gépeléskor NEM re-renderelődik az egész komponens.

5.2 handleSubmit() – validáció + submit

```
async function onSubmit(data) {
  await login(data.email, data.password);
}

<form onSubmit={handleSubmit(onSubmit)}>
```

Kliens oldali vs szerver oldali validáció

A register() szabályai kliens oldalon validálnak – gyors visszajelzés. A Laravel Form Request validáció mindig megmarad védelemként, mert a kliens oldali megkerülhető.

6. ProtectedRoute – védett útvonalak

```
function ProtectedRoute({ children }) {
  const { user, loading } = useAuth();
  if (loading) return <div>Betöltés...</div>;
  if (!user) return <Navigate to='/login' replace />;
  return children;
}
```

A három állapot logikája

1) loading: még nem tudjuk, várunk. 2) nincs user: átirányítás login-ra. 3) van user: megjelenítjük a védett tartalmat. A sorrend fontos a villanás elkerülése miatt.

7. Laravel oldali catch-all route – SPA routing

Amikor a React Router kezel egy URL-t, az csak kliens oldalon történik. Ha valaki közvetlenül beírja az URL-t vagy F5-öt nyom, a böngésző a szervertől kéri le azt az oldalt – de a Laravel nem ismeri a React Router route-jait.

```
Route::get('/{any}', function () {  
    return view('welcome');  
})->where('any', '.*');
```

Mit csinál ez a route?

Bármilyen URL-t kérünk le, mindig ugyanazt a welcome.blade.php-t kapjuk – ez betölti a React alkalmazást, és onnantól a React Router jeleníti meg a megfelelő komponenst.

8. Hibakeresési napló

Hiba	Ok	Megoldás
JSX parse error app.js-ben	Fájl .js volt .jsx helyett	Átnevezés app.jsx-re
404 a /login URL-nél	Laravel nem ismeri a React route-okat	Catch-all route web.php-ban
handlesubmit is not a function	Elgépelés: kisbetűs handlesubmit	Javítás handleSubmit-re
Nincs hálózati kérés login-nál	Interceptor nem adott vissza config-ot	return config; hozzáadása
createRoot() called twice	Vite HMR mellékhatása	Ártalmatlan dev figyelmeztetés

9. Frontend mappa struktúra

```
resources/js/  
  api/axios.js           - Axios instance + interceptorok  
  context/AuthContext.jsx - bejelentkezett user globálisan  
  hooks/useAuth.js       - AuthContext elérése egyszerűen  
  pages/LoginPage.jsx    - bejelentkezési oldal  
  routes/AppRoutes.jsx   - React Router + ProtectedRoute  
  app.jsx                - belépési pont
```

10. Összefoglaló – a bejelentkezési folyamat

1. Form kitöltés -> React Hook Form validál
2. handleSubmit -> onSubmit(data)
3. onSubmit -> useAuth().login(email, password)
4. AuthContext login() -> api.post('/auth/login')
5. Axios interceptor hozzácsatolja a headereket
6. Laravel validál, token generál
7. Válasz: { token, user }
8. localStorage + setUser(user)
9. navigate('/dashboard')
10. ProtectedRoute megjeleníti a Dashboardot
11. Minden további kérésnél a token automatikus

– School Management App fejlesztési dokumentáció –